

Form Tag Helpers in ASP.NET Core



<Form> е достатъчен за създаване на форма. Останалите са за контролни елементи.

Form Tag Helpers in ASP.NET Core

The screenshot shows a web form with a dark header bar containing 'List' and 'Create' links. Below the header, there are three input fields: 'Name', 'Email', and 'Department' (a dropdown menu with 'Choose...' selected). A blue 'Create' button is at the bottom left.

Форма с всички необходими данни, които трябва да бъдат прихванати за създаването на нов служител. Досега „Create“ не работи при клик – 404, сега ще създадем action Create in HomeController + Create View in Home Folder.

The screenshot shows two files in Visual Studio. The top file is 'Create.cshtml' with the following code:

```
1 @model Employee
2
3 @{
4     ViewBag.Title = "Create Employee";
5 }
6 <form asp-controller="Home" asp-action="Create" method="post">
7
8 </form>
```

The bottom file is 'HomeController.cs' with the following code:

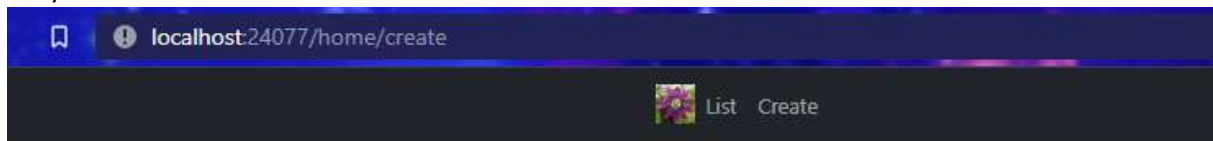
```
public IActionResult Create()
{
    return View();
}
```

Веднага включваме първо @model = Strongly type view:

- * Intellisense
- * autocomplete

* compile-time error checking.

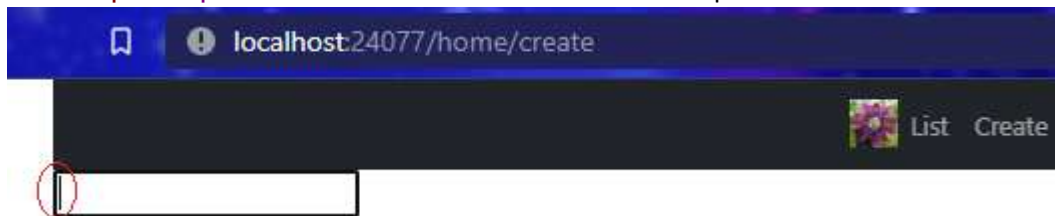
Резултат:



ViewSource: `<form method="post" action="/Home/Create">`

В този адрес се изпълнява Action method in HomeController, който извиква съответното Create View, като при попълването на формата и натискането на бутона, се изпълнява метод post, чрез който се създава нов служител и това става в същия URL. Най-интересният момент тук е, че дори от таг `<form asp-controller="Home" asp-action="Create" method="post">` да бъдат премахнати `asp-controller="Home" asp-action="Create"`, всичко това ще бъде обслужено пак в същия /home/create, защото по този начин е инициализирано в HomeController и Create View.

`<input asp-for="Name" />` С което вече имаме input елемент.



```
<form method="post" action="/Home/Create">
  <input type="text" id="Name" name="Name" value="" />
```

т.е. въвежда се текст, който има value = empty string, атрибути id = name = "Name", заради процес, който се нарича Model Binding.

`<label asp-for="Name"></label>` - полето се нарича Name, то автоматично ще бъде показано като текст Name и свързано с input след него. Това означава, че когато се кликне върху Name, автоматично ще се активира полето за въвеждане на текст от input след него, като курсорът се появи.



```

37 <form method="post" action="/Home/Create">
38   <label for="Name">Name</label>
39   <input type="text" id="Name" name="Name" value="" />
40 </form>

```

```

<div>
  <label asp-for="Name"></label>
  <input asp-for="Name" />
</div>

```

```

1 @model Employee
2
3 @{
4   ViewBag.Title = "Create Employee";
5 }
6 <form asp-controller="Home" asp-action="Create" method="post">
7   <div>
8     <label asp-for="Name"></label>
9     <input asp-for="Name" />
10  </div>
11  <div>
12    <label asp-for="Email"></label>
13    <input asp-for="Email" />
14  </div>
15  <div>
16    <label asp-for="Department"></label>
17    <select asp-for="Department"></select>
18  </div>
19 </form>

```

<select> има няколко начина за въвеждане на опциите:

- * html – hard code
- * data from DB table
- * enum

Ние ще използваме Enum: Models->add-> Class-> Dept.cs, създавайки нов сложен тип данни.

```

Dept.cs
1 namespace EmployeeManagement.Models
2 {
3   public enum Dept
4   {
5     None,
6     HR,
7     IT,
8     Payroll
9   }
10 }

```

```

namespace EmployeeManagement.Models
{
    public enum Dept
    {
        None,
        HR,
        IT,
        Payroll
    }
}

```

Въвеждаме го там, където ни трябваше в **<select>**

```

<div>
  <label asp-for="Department"></label>
  <select asp-for="Department" asp-items="Html.GetEnumSelectList<Dept>()"></select>
</div>

```

System.Collections.Generic.IEnumerable<Microsoft.AspNetCore.Mvc.Rendering.SelectListItem>
Microsoft.AspNetCore.Mvc.TagHelpers.SelectTagHelper.Items
A collection of Microsoft.AspNetCore.Mvc.Rendering.SelectListItem objects used to populate the <select> element with <optgroup> and <option> elements.

```

<select asp-for="Department" asp-items="Html.GetEnumSelectList<Dept>()"></select>

```

Целият код на Create:

@model Employee

@{

ViewBag.Title = "Create Employee";

}

<form asp-controller="Home" asp-action="Create" method="post">

<div>

<label asp-for="Name"></label>

<input asp-for="Name" />

</div>

<div>

<label asp-for="Email"></label>

<input asp-for="Email" />

</div>

<div>

<label asp-for="Department"></label>

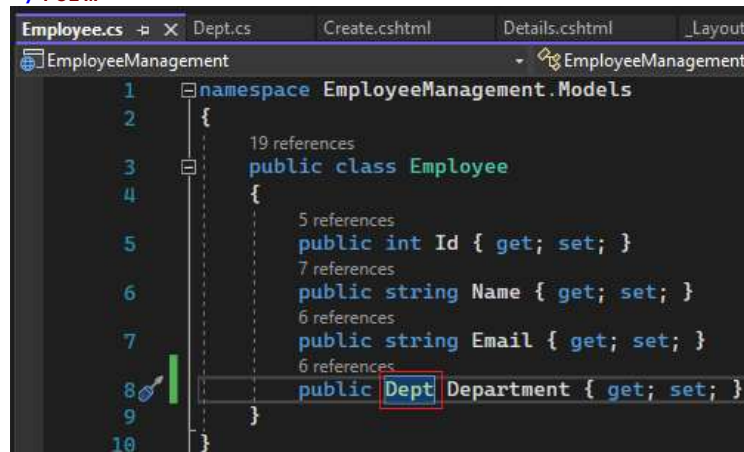
<select asp-for="Department"

asp-items="Html.GetEnumSelectList<Dept>()">

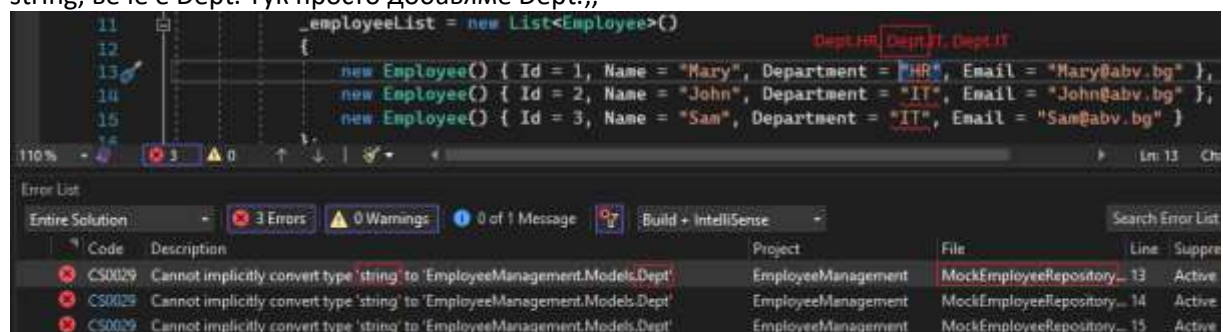
</select>

</div>

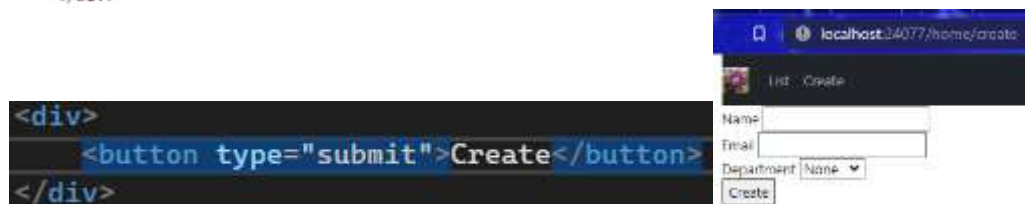
</form>



При изпълнение получаваме 3 грешки в MockEmployeeRepository, които при клик ни отвеждат в реда на кода. Причината е, че string не може да се преобразува в Dept. Преди Department беше string, вече е Dept. Тук просто добавяме Dept.,,



```
public MockEmployeeRepository() // ctor
{
    _employeeList = new List<Employee>()
    {
        new Employee() { Id = 1, Name = "Mary", Department = Dept.HR,
Email = "Mary@abv.bg" },
        new Employee() { Id = 2, Name = "John", Department = Dept.IT,
Email = "John@abv.bg" },
        new Employee() { Id = 3, Name = "Sam", Department = Dept.IT,
Email = "Sam@abv.bg" }
    };
}
```



```

<div>
    <button type="submit">Create</button>
</div>

```

Оформяме с bootstrap:

@model Employee

@{

ViewBag.Title = "Create Employee";

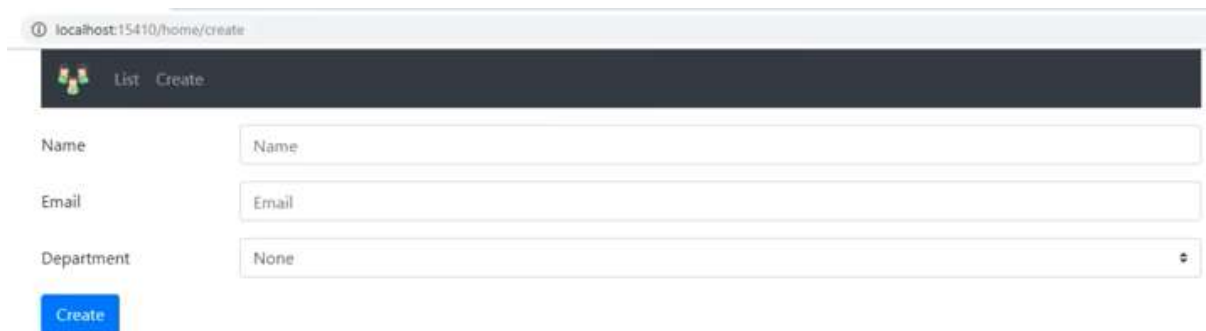
}

```

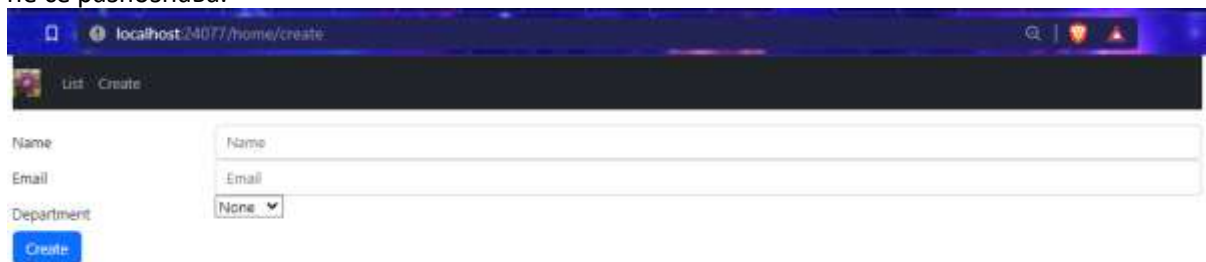
<form asp-controller="Home" asp-action="Create" method="post" class="mt-3">
    <div class="form-group row">
        <label asp-for="Name" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10">
            <input asp-for="Name" class="form-control" placeholder="Name"/>
        </div>
    </div>
    <div class="form-group row">
        <label asp-for="Email" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10">
            <input asp-for="Email" class="form-control" placeholder="Email"/>
        </div>
    </div>
    <div class="form-group row">
        <label asp-for="Department" class="col-sm-2 col-form-label"></label>
        <div class="col-sm-10">
            <select asp-for="Department" class="custom-select mr-sm-2"
                asp-items="Html.GetEnumSelectList<Dept>()">
            </select>
        </div>
    </div>
    <div class="form-group row">
        <div class="col-sm-10">
            <button type="submit" class="btn btn-primary">Create</button>
        </div>
    </div>
</form>

```

Това трябва да се получи:

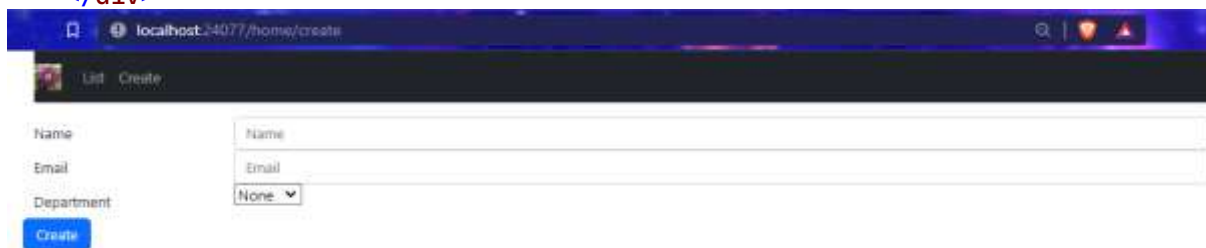


Но при мен Department дава отклонение, понеже `class="custom-select"` не се разпознава.



Копирах и кода от блога на tutoriela и резултатът е отново същият. Направи ми впечатление, че има два последователни `<div>` за бутона, затова опитах да ги обединя в един `<div class=`, което даде отклонение в изместването на бутона вляво и нагоре

```
<div class="form-group row col-sm-10">  
  <button type="submit" class="btn btn-primary">Create</button>  
</div>
```

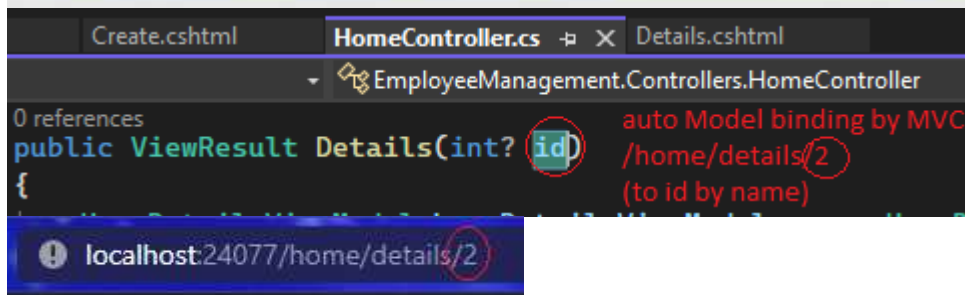


Значи проблем остана при `class="custom-select"` за Department.

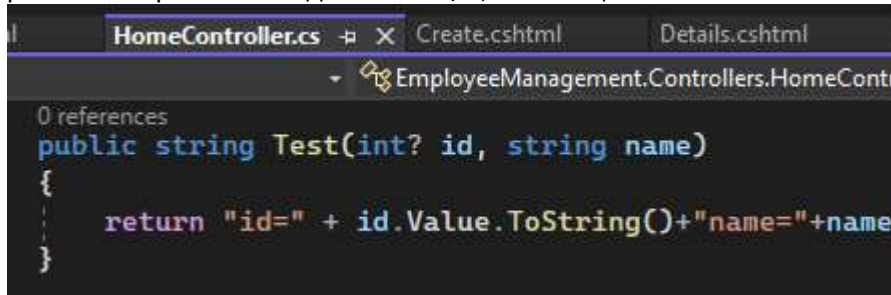
ASP.NET Core Model Binding

Model binding maps data in an HTTP request to controller action method parameters

The **action parameters** may be **simple** types such as integers, strings, etc or **complex** types like Customer, Employee, Order etc

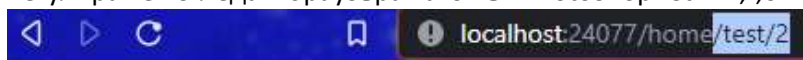


С това автоматично прихващане и разпознаване, MVC ни спестява много код. Демонстративно изменяме кода, включвайки нов параметър, но аз предпочитам да не закоментирам и разкоментирам блок код с Ctrl+K+C/U, затова ще го напиша в Test action:



```
public string Test(int? id, string name)
{
    return "id=" + id.Value.ToString() + "name=" + name;
}
```

Регулираме изгледа в брауъра като във PhotoShop: Ctrl++, -, 0.



id=2name= към момента name=0

В URL адреса може да добавим string query: /home/details/2?name=abv

localhost:24077/home/test/2?name=abv

id= 2 name= abv

localhost:24077/home/test/2?name1=abv

id= 2 name=

localhost:24077/home/test/2?name=abv&id=5

id= 2 name= abv последователност: прихваща първата стойност

ASP.NET Core Model Binding

To bind the request data to the controller action method parameters, model binding looks for data in the HTTP request



Премахваме стойността 2 и прихваща id=5, нищо, че е зададено след name:

localhost:24077/home/test/?name=abv&id=5

id= 5 name= abv

За да създадем нов служител, използваме action Create, който трябва да се изпълнява 2 пъти: единият за get, другият за post в рамките на 1 и същи URL адрес:

В противен случай ще отчете грешка, че има 2 Create, които неразлично изпълняват условията (Ambiguous – двусмислие)

localhost:24077/home/create

An unhandled exception occurred while processing the request.

AmbiguousActionException: Multiple actions matched. The following actions matched route data and had all constraints satisfied:

EmployeeManagement.Controllers.HomeController.Create (EmployeeManagement)
EmployeeManagement.Controllers.HomeController.Create (EmployeeManagement)

[HttpGet]

```
public IActionResult Create()  
{  
    return View();  
}
```

[HttpPost]

```
public IActionResult Create()
```

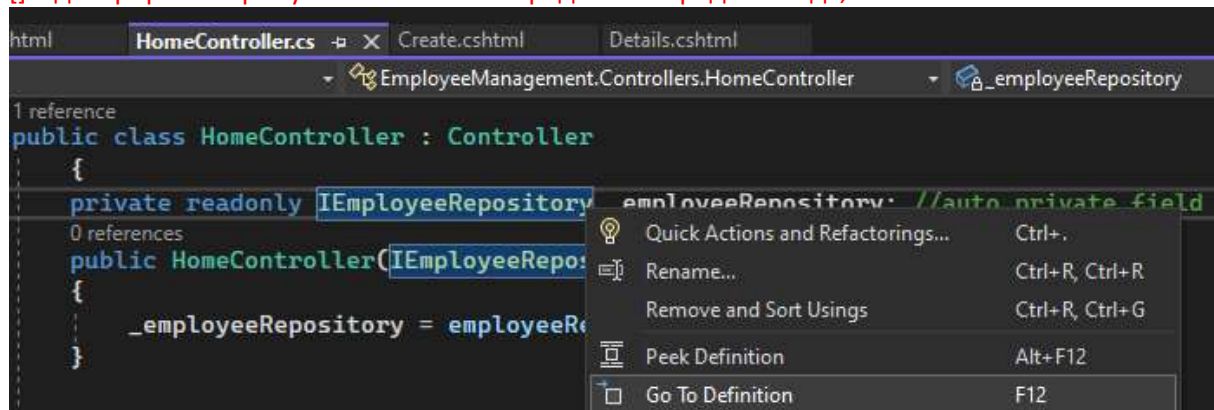


```

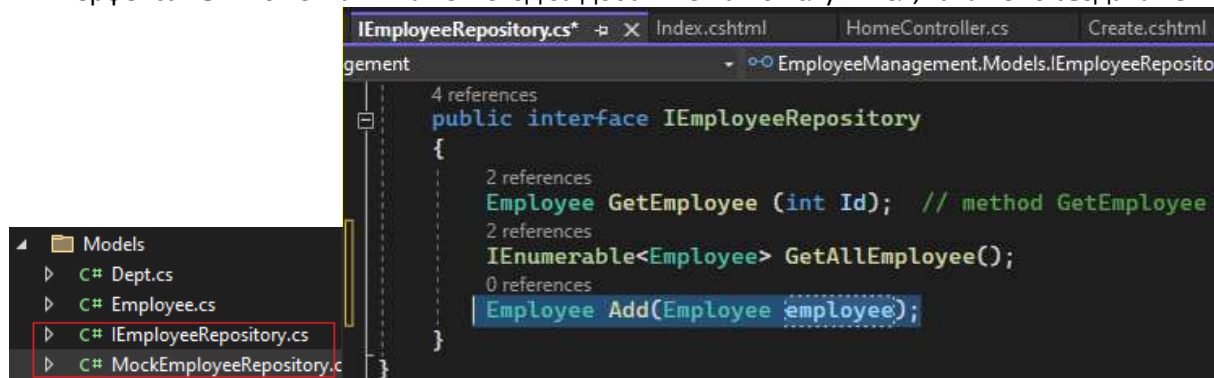
{
    return View();
}

```

[] – декорираме атрибутите винаги непосредствено преди метода, за който се отнасят.

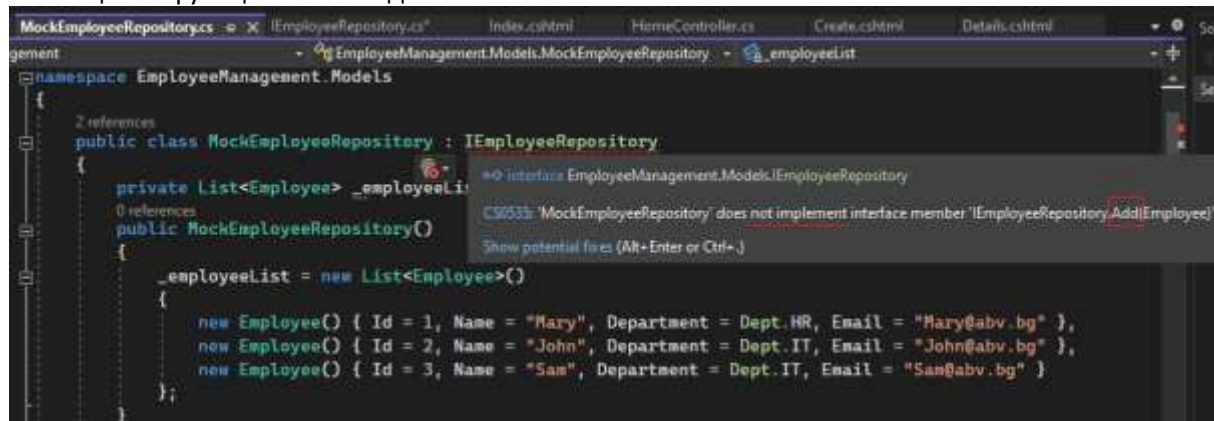


В интерфейса към момента нямаме метод за добавяне на нов служител, така че го създаваме:

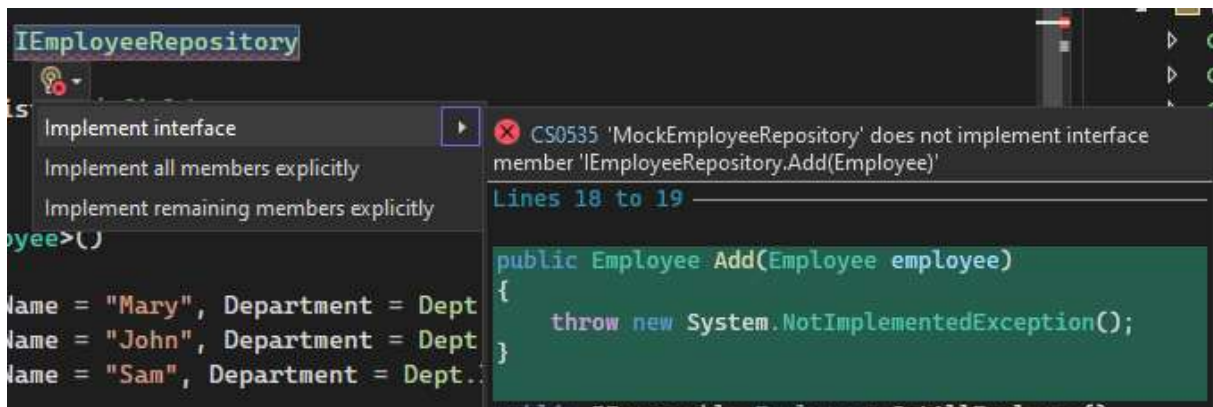


Employee Add(Employee employee);

Следва да създадем имплементация за него в MockEmployeeRepository, където се изпълнява почти целия функционален код.



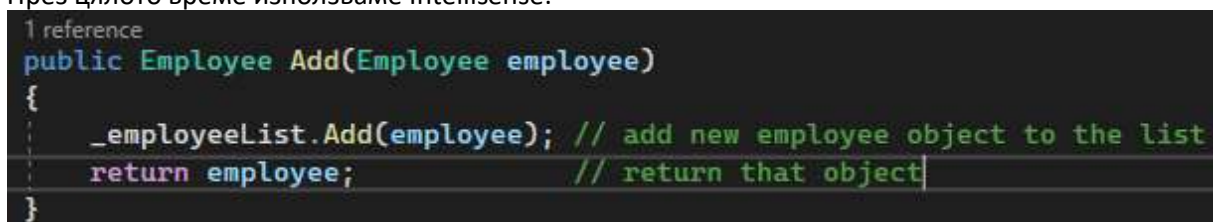
Автокорекцията е с най-простата имплементация чрез употребата на изключение:



Ние обаче искаме да добавим нов член към съществуващия преди него лист, т.е. искаме да добавим employee -> _employeeList

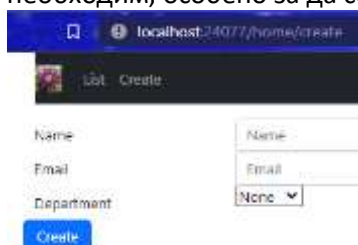


През цялото време използваме Intellisense:



```
public Employee Add(Employee employee)
{
    _employeeList.Add(employee); // add new employee object to the list
    return employee;             // return that object
}
```

Във формата за попълване има само Name, Email, Department, но скрит остава id, който също е необходим, особено за да се добави към горния списък:



За целта използваме Max method, като към най-голямото Id от списъка +1 и новата му стойност се предава като Id на новият employee object.

```
MockEmployeeRepository.cs* Index.cshtml HomeController.cs Create.cshtml Details.cshtml
EmployeeManagement.Models.MockEmployeeRepository Add(Employee employee)

_employeeList = new List<Employee>()
{
    new Employee() { Id = 1, Name = "Mary", Department = Dept.HR, Email = "Mary@abv.bg" },
    new Employee() { Id = 2, Name = "John", Department = Dept.IT, Email = "John@abv.bg" },
    new Employee() { Id = 3, Name = "Sam", Department = Dept.IT, Email = "Sam@abv.bg" }
};

1 reference
public Employee Add(Employee employee)
{
    employee.Id = _employeeList.Max(e => e.Id)+1; // max id form the list + 1 = new id
    _employeeList.Add(employee); // add new employee object to the list
    return employee; // return that object
}
```

```
public Employee Add(Employee employee)
{
    employee.Id = _employeeList.Max(e => e.Id)+1; // max id + 1 = new id
    _employeeList.Add(employee); // add new employee object to the list
    return employee; // return that object
}
```

```
MockEmployeeRepository.cs HomeController.cs* MockEmployeeRepository.cs Index.cshtml Create.cshtml Det
EmployeeManagement.Controllers.HomeController _employeeRepository
1 reference
public class HomeController : Controller
{
    private readonly IEmployeeRepository _employeeRepository; //auto private field
    0 references
    public HomeController(IEmployeeRepository employeeRepository)
    {
        _employeeRepository = employeeRepository;
    }

    0 references
    public ViewResult Index()
    {
        var model = _employeeRepository.GetAllEmployee();
        return View(model);
    }

    0 references
    public ViewResult Details(int? id)
    {
        HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel()
        {
            Employee = _employeeRepository.GetEmployee(id ?? 1), //c# colesing operator
            PageTitle = "Employee Details"
        };
        return View(homeDetailsViewModel);
    }
}
```

Като погледнем, към момента всички action methods използват `_employeeRepository` от частното поле, създадено още в началото на `HomeController` за тази цел. Използваме инжектирания `IEmployeeRepository` и добавяме този `_employeeRepository`:

```
_employeeRepository.Add(employee);
```

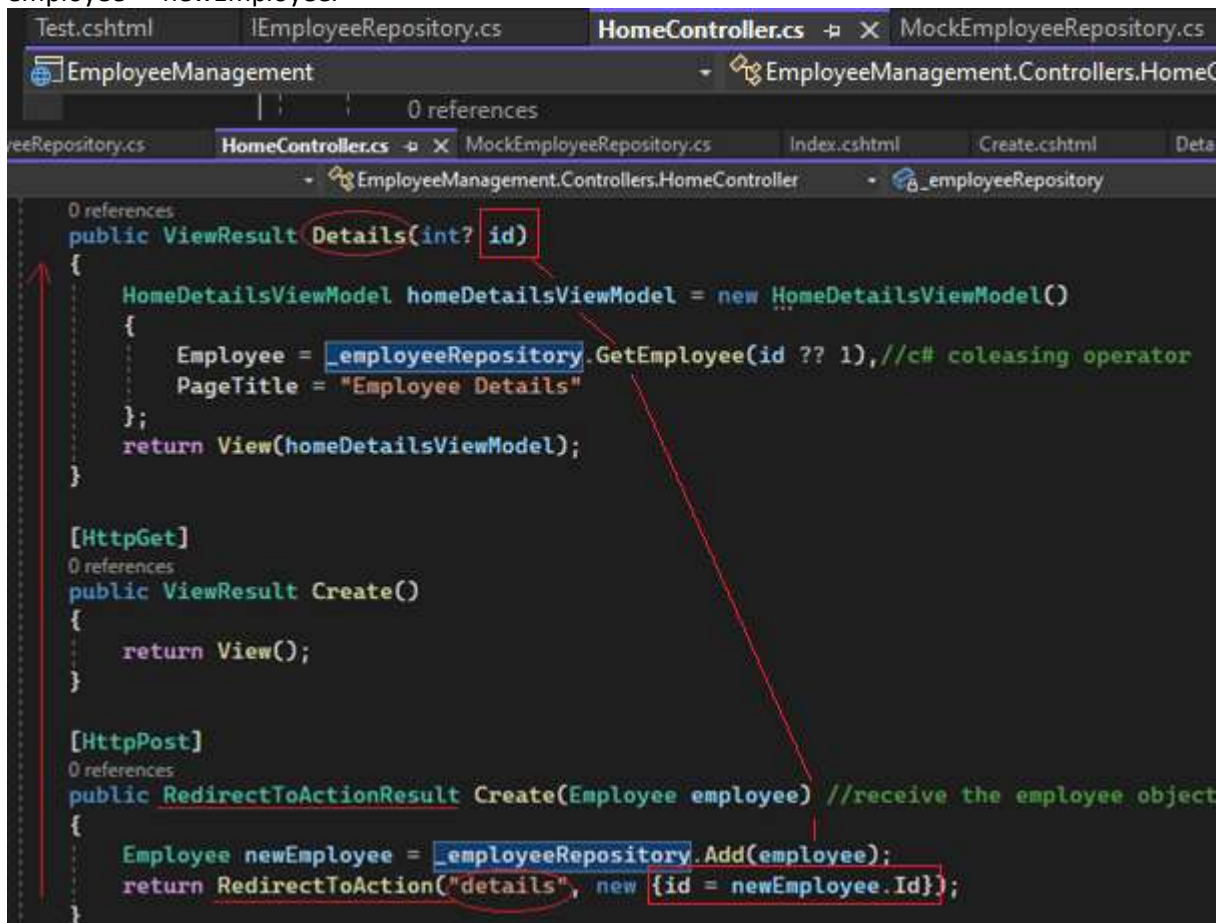
като му прехвърляме `employee` (т.е. той прихваща `employee`), на който вече сме присвоили новото `id`.

```
Employee newEmployee = _employeeRepository.Add(employee);
```

Съхраняваме този нов служител от клас `Employee` в нова променлива, която наричаме

newEmployee.

След това пренасочваме (redirect) потребителя към Details action, за да види новите данни на новия служител, който току-що сме създали. Също му подаваме ново Id, което се взема от employee -> newEmployee.



```
0 references
public ViewResult Details(int? id)
{
    HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel()
    {
        Employee = _employeeRepository.GetEmployee(id ?? 1), //c# coalescing operator
        PageTitle = "Employee Details"
    };
    return View(homeDetailsViewModel);
}

[HttpGet]
0 references
public ViewResult Create()
{
    return View();
}

[HttpPost]
0 references
public RedirectToActionResult Create(Employee employee) //receive the employee object
{
    Employee newEmployee = _employeeRepository.Add(employee);
    return RedirectToAction("details", new {id = newEmployee.Id});
}
```

Т.е. искаме в HomeController да се изпълни Create, която чрез извикване на Add да създаде нов служител и като се попълни формата ѝ + автоматично новото id, да съхрани новите данни, като ги добави в _employeeRepository, а после да прехвърли потребителя към details. Самата функция RedirectToAction изисква да се подаде името на action метода, към който пренасочваме и променлива, която ще се подаде, а според характеристиките на details, се подава id. Ние подаваме id чрез създаването на нов анонимен {} обект.

new {anonymus object} include a property with name id, and the value for this is come from the property Id of this newEmployee object.

RedirectToAction изисква тип RedirectToActionResult.

```
[HttpGet]
public ViewResult Create()
{
    return View();
}

[HttpPost]
public RedirectToActionResult Create(Employee employee)
{
    Employee newEmployee = _employeeRepository.Add(employee);
    return RedirectToAction("details", new {id = newEmployee.Id});
}
```

}

localhost:24077/home/create

List Create

Name David

Email david@abv.bg

Department IT

Create

localhost:24077/Home/Details/4

List Create

David



Employee ID:
EmployeeManagement.Models.E
mployee

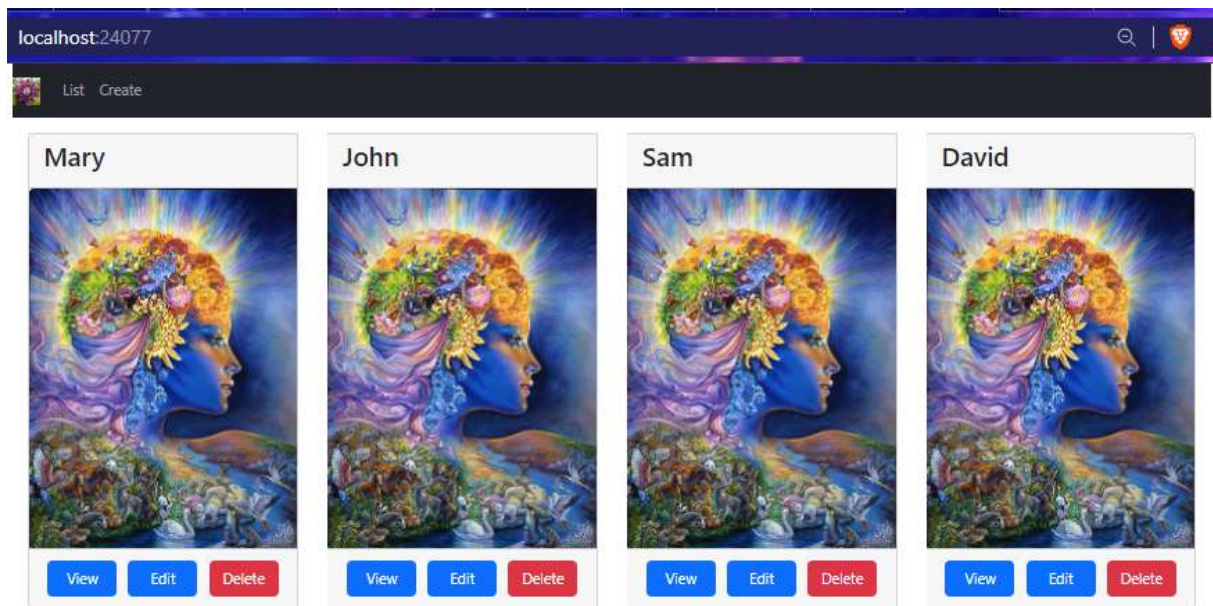
Employee Email: david@abv.bg
Employee Department: IT

Back Edit Delete

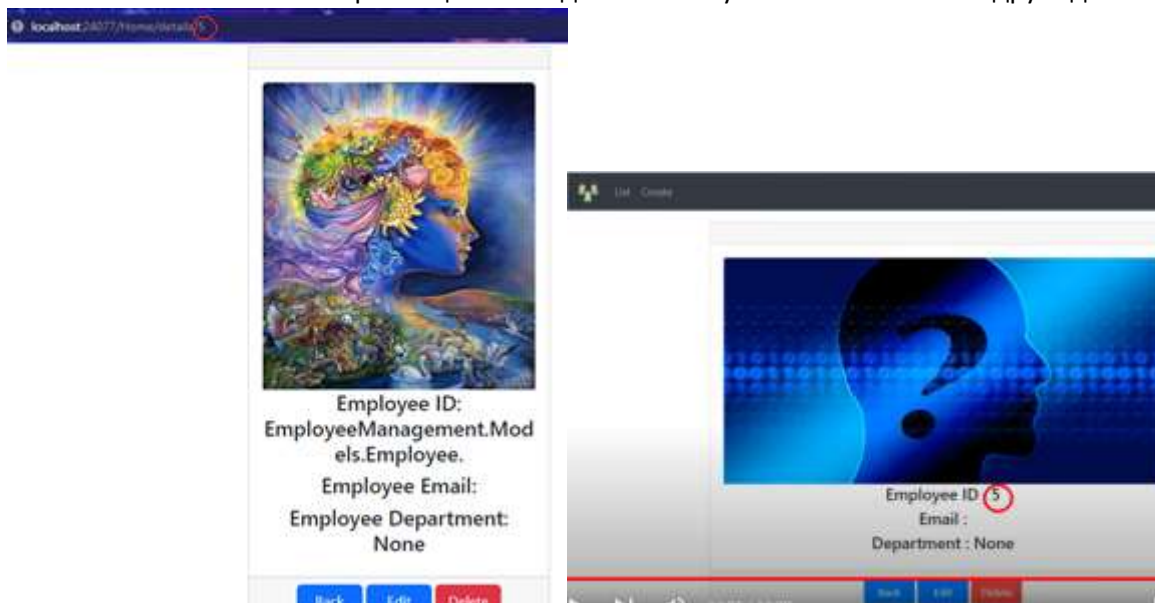
Имената, които задаваме на формата са много важни – те трябва да съвпадат с предварително дефинираните в Employee (Name-Name, Email-Email, Department-Department), за да може да се осъществи адекватен мапинг.

```
namespace EmployeeManagement.Models
{
    25 references
    public class Employee
    {
        8 references
        public int Id { get; set; }
        7 references
        public string Name { get; set; }
        6 references
        public string Email { get; set; }
        6 references
        public Dept Department { get; set; }
    }
}

<label class="col-sm-2 col-form-label" for="Name">Name</label>
<div class="col-sm-10">
    <input class="form-control" placeholder="Name" type="text" id="Name" name="Name" value="" />
</div>
</div>
<div class="form-group row">
    <label class="col-sm-2 col-form-label" for="Email">Email</label>
    <div class="col-sm-10">
        <input class="form-control" placeholder="Email" type="text" id="Email" name="Email" value="" />
    </div>
</div>
<div class="form-group row">
    <label class="col-sm-2 col-form-label" for="Department">Department</label>
    <div class="col-sm-10">
        <select class="custom-select mr-sm-2" data-val="true" data-val-required="The Department field is required." id="Department" name="Department">
```

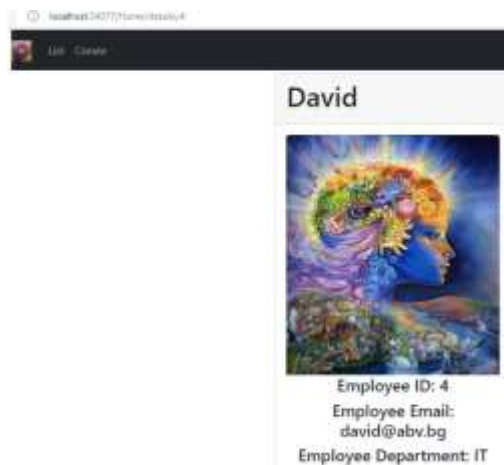
Вече създаваме нови служители, но нямаме валидация, затова може да попълним с празни полета и пак автоматично прихваща id и създава нови служители без имена и други данни.



Вижда се, че в браузъра id = 5, но при мен не се изписва. Когато попълня формата в единия браузър, промените се виждат и в другия.

Намерих си грешката – без да искам нещо съм натиснала и изтрила .ID, .Email от Details

```
<h4>Employee ID: @Model.Employee.Id</h4>
<h4>Employee Email: @Model.Employee.Email</h4>
<h4>Employee Department: @Model.Employee.Department</h4>
```



Към момента може да добавяме служител без
попълнени полета. В следващия урок ще забраним това с model validation.

